BẢN TỔNG HỢP VÀ PHÂN TÍCH CHUYÊN SÂU PAPER

AI Agent Systems for Supply Chains: Structured Decision Prompts and Memory Retrieval

Tác giả: Konosuke Yoshizato, Kazuma Shimizu, Ryota Higa, Takanobu Otsuka

Nagoya Institute of Technology, Nhật Bản — Kỷ yếu nghiên cứu **arXiv:2602.05524** (02/2026)

Lĩnh vực: Multiagent Systems (`cs.MA`) & Artificial Intelligence (`cs.AI`) | DOI: 10.48550/arXiv.2602.05524

—

1. Bối Cảnh & Nghịch Lý Ra Quyết Định Tồn Kho Của LLM Agent

Trong chuỗi cung ứng hiện đại và quản lý tài sản doanh nghiệp, bài toán kiểm soát tồn kho (Inventory Control) là mạch máu quyết định chi phí vận hành và chất lượng dịch vụ. Các hệ thống truyền thống vận hành dựa trên hai trường phái:

1. **Phương pháp nghiên cứu vận hành cổ điển (Operations Research - OR):** Sử dụng các chính sách heuristic như (s, S) policy, Order-Up-To Level (OUT), hoặc Economic Order Quantity (EOQ). Các giải thuật này tối ưu hoàn hảo dưới giả định phân phối nhu cầu cố định (Stationary Poisson/Normal), nhưng trở nên tê liệt khi thị trường đối mặt với biến động bất ngờ (Demand Shocks) hoặc dữ liệu phi cấu trúc.
2. **Học tăng cường sâu (Deep Reinforcement Learning - DRL):** Các mô hình PPO, DQN có khả năng học chính sách thích nghi nhưng là "hộp đen" tuyệt đối, đòi hỏi hàng triệu bước mô phỏng và cực kỳ nhạy cảm với hiện tượng Reward Misalignment khi môi trường thay đổi phân phối.

Sự bùng nổ của Large Language Models (LLMs) mở ra kỷ nguyên Multi-Agent Systems (MAS) có khả năng lập luận ngôn ngữ tự nhiên. Tuy nhiên, Yoshizato et al. (02/2026) chỉ ra hai nghịch lý nghiêm trọng khi ứng dụng LLM Agent vào quyết định số học chuỗi cung ứng:

- **Hiện tượng mù độ trễ (Lead-Time Blindness) & Hiệu ứng roi da (Bullwhip Effect):** Khi hàng đặt có độ trễ $L > 0$ chu kỳ mới về tới kho, LLM nếu chỉ nhận prompt đơn giản sẽ liên tục đặt hàng dồn dập ở các chu kỳ tiếp theo do thấy kho vẫn thiếu hàng. Kết quả là tạo ra làn sóng tồn kho khổng lồ sau khi hàng cập bến, khuếch đại sai lệch đơn hàng theo cấp số nhân dọc chuỗi cung ứng.
- **Thiếu bộ nhớ kinh nghiệm thực chứng (Experience Amnesia):** Các mô hình Agent thông thường hoạt động phi trạng thái (Stateless); chúng lặp lại các sai lầm điều phối tương tự nhau qua từng chu kỳ mà không thể đúc rút kinh nghiệm từ các kịch bản tương đồng trong quá khứ.

LÝ THUYẾT CỐT LÕI: Đột Phá Của Yoshizato et al. (2026)

Nghiên cứu của Yoshizato et al. giải quyết triệt để hai điểm nghẽn trên bằng giải pháp kết hợp kép:

1. **Structured Decision Prompts (Prompt quyết định có cấu trúc):** Mã hóa tường minh quy trình suy luận từng bước gắn chặt với cơ chế tồn kho an toàn (*Safe-stock strategy*),

ép LLM phải tính toán đúng vị thế tồn kho khả dụng gồm cả các đơn hàng đang trên đường vận chuyển (*Pipeline orders*).

2. **Kiến trúc Agent AIM-RM (Agent for Inventory Management with Retrieval Memory):** Tích hợp bộ nhớ kinh nghiệm hồi cứu dựa trên khớp độ đo tương đồng (*Similarity Matching*), cho phép Agent truy xuất các tình huống lịch sử tương tự và chính sách đặt hàng thành công để làm mẫu suy luận trong ngữ cảnh (*In-Context Exemplars*).

—

2. Hình Thức Hóa Toán Học: Động Học Tồn Kho & Không Gian Trạng Thái

Yoshizato et al. xây dựng mô hình toán học chặt chẽ cho bài toán bổ sung tồn kho tuần hoàn nhiều chu kỳ ($t = 1, 2, \ldots, T$).

MÔ HÌNH HÓA TOÁN HỌC: 1. Phương Trình Chuyển Trạng Thái Hệ Thống

Tại mỗi chu kỳ t, hệ thống diễn ra trình tự sự kiện sau:

1. Lô hàng đặt từ chu kỳ $t - L$ (với thời gian trễ L) cập bến kho: lượng nhận là q_{t-L}.
2. Phát sinh nhu cầu thực tế d_t từ người dùng hoặc thị trường.
3. Cập nhật mức tồn kho vật lý tại kho I_t và lượng đơn hàng nợ/chờ xử lý B_t (Backorders):

$$I_{t+1} = \max\left(0,\ I_t + q_{t-L} - d_t\right) \tag{1}$$
$$B_{t+1} = \max\left(0,\ B_t + d_t - (I_t + q_{t-L})\right) \tag{2}$$

4. Đại lý quyết định số lượng đặt hàng mới $q_t \geq 0$.

MÔ HÌNH HÓA TOÁN HỌC: 2. Hàm Mục Tiêu Tối Thiểu Hóa Tổng Chi Phí Vận Hành

Chính sách điều phối π của Agent có mục tiêu tìm chuỗi quyết định $\{q_t\}_{t=1}^{T}$ sao cho tối thiểu hóa hàm kỳ vọng tổng chi phí chu kỳ $J(\pi)$:

$$\min_{\pi}\ J(\pi) = \mathbb{E}\left[\sum_{t=1}^{T} \gamma^t \left(c_h \cdot I_t + c_p \cdot B_t + c_o \cdot \mathbb{I}(q_t > 0)\right)\right] \tag{3}$$

Trong đó:

- c_h: Đơn giá chi phí lưu kho trên một đơn vị sản phẩm trong một chu kỳ (Holding Cost).
- c_p: Đơn giá phạt do thiếu hụt hàng/đứt chuỗi cung ứng (Backorder Penalty Cost, thường $c_p \gg c_h$).
- c_o: Chi phí cố định cho mỗi lần phát lệnh đặt hàng (Fixed Ordering Cost).
- $\gamma \in (0, 1]$: Hệ số chiết khấu theo thời gian.

MÔ HÌNH HÓA TOÁN HỌC: 3. Khái Niệm Vị Thế Tồn Kho Khả Dụng (Inventory Position)

Điểm then chốt để loại bỏ hiện tượng đặt trùng đơn và mù độ trễ là khái niệm **Vị thế tồn kho** IP_t:

$$IP_t = I_t + \sum_{k=1}^{L} q_{t-k} - B_t \tag{4}$$

Trong đó $\sum_{k=1}^{L} q_{t-k}$ là tổng lượng hàng đang trên đường vận chuyển (*Pipeline Inventory*). Mọi công thức quyết định tối ưu phải dựa trên IP_t thay vì chỉ nhìn vào tồn kho vật lý I_t.

—

3. Kiến Trúc Cốt Lõi: Structured Prompts & Mô Hình AIM-RM

3.1 Cấu Trúc Stepwise Prompt Kết Hợp Chiến Lược Safe-Stock

Thay vì yêu cầu LLM đưa ra số lượng đặt hàng bằng ngôn ngữ tự do, nghiên cứu xây dựng cấu trúc prompt ràng buộc chặt chẽ 4 bước tính toán xác định:

1. **Bước 1 (Dự phóng nhu cầu thời gian trễ):** Ước lượng nhu cầu kỳ vọng trong toàn bộ khoảng thời gian từ lúc đặt hàng đến khi hàng về kho:

$$\hat{D}_{L+1} = \hat{\mu}_d \cdot (L+1) \tag{5}$$

2. **Bước 2 (Xác định tồn kho an toàn động - Dynamic Safety Stock):**

$$S_{\text{safe}} = z_\alpha \cdot \hat{\sigma}_d \cdot \sqrt{L+1} \tag{6}$$

   Trong đó z_α là giá trị phân vị tương ứng với mức độ thỏa mãn dịch vụ mong muốn (Service Level $1-\alpha$, ví dụ $z_{0.95} = 1.645$).

3. **Bước 3 (Tính toán mức trần mục tiêu - Order-Up-To Level):**

$$S^* = \hat{D}_{L+1} + S_{\text{safe}} \tag{7}$$

4. **Bước 4 (Quyết định số lượng đặt hàng):** Bù đắp khoảng chênh lệch giữa mức trần mục tiêu và vị thế tồn kho thực tế:

$$q_t^* = \max\left(0,\ S^* - IP_t\right) = \max\left(0,\ \hat{D}_{L+1} + S_{\text{safe}} - \left(I_t + \sum_{k=1}^{L} q_{t-k} - B_t\right)\right) \tag{8}$$

3.2 Cơ Chế Hồi Cứu Kinh Nghiệm Tương Đồng (AIM-RM)

Kiến trúc AIM-RM (*Agent for Inventory Management with Retrieval Memory*) được thiết kế nhằm thích ứng với các kịch bản phân phối nhu cầu phi tĩnh (Non-stationary demand) và cú sốc thị trường:

1. **Lưu trữ dữ liệu tập sự (Episode Storage):** Mỗi chu kỳ k trong quá khứ được số hóa thành một tuple trạng thái - hành động - hệ quả:

$$e_k = (\mathbf{v}_k,\ q_k,\ c_k,\ \text{Outcome}_k) \tag{9}$$

   Trong đó vector đặc trưng trạng thái $\mathbf{v}_k \in \mathbb{R}^m$ bao gồm:

$$\mathbf{v}_k = [\hat{\mu}_d,\ \hat{\sigma}_d,\ I_k,\ B_k,\ q_{k-1},\ \ldots,\ q_{k-L},\ \text{Trend}_k]^T \tag{10}$$

2. **Truy xuất tương đồng Cosine (Cosine Similarity Retrieval):** Khi đứng trước chu kỳ mới t, hệ thống tính toán vector đặc trưng hiện tại $\mathbf{v}_t$ và quét trong kho bộ nhớ $\mathcal{M}$:

$$\text{Sim}(\mathbf{v}_t, \mathbf{v}_k) = \frac{\mathbf{v}_t \cdot \mathbf{v}_k}{\|\mathbf{v}_t\|_2 \, \|\mathbf{v}_k\|_2} \tag{11}$$

3. **Lựa chọn kịch bản mẫu thành công (Top-k Low-Cost Exemplars):** AIM-RM không chọn ngẫu nhiên mà lọc ra top-k kịch bản có độ tương đồng cao nhất đồng thời có tổng chi phí chu kỳ c_k ở phân vị thấp nhất:

$$\mathcal{E}^* = \arg \max_{e_k \in \mathcal{M},\, c_k < \tau} \text{Sim}(\mathbf{v}_t, \mathbf{v}_k) \tag{12}$$

4. **Bơm ngữ cảnh và tự hiệu chỉnh (In-Context Adaptation):** Các kịch bản $\mathcal{E}^*$ được chèn trực tiếp vào Prompt của LLM, chỉ dẫn Agent quan sát cách xử lý trong quá khứ: *"Trong tình huống tương tự khi nhu cầu tăng đột biến 150%, việc nâng tạm thời hệ số an toàn z_α từ 1.65 lên 2.10 đã ngăn chặn đứt hàng mà không làm bùng nổ chi phí lưu kho."*

—

4. Kết Quả Thực Nghiệm & Đóng Góp Khoa Học

THỰC NGHIỆM ĐỐI CHỨNG: Hiệu Năng Vượt Trội Trên Các Kịch Bản Biến Động

Yoshizato et al. tiến hành thử nghiệm toàn diện trên mô hình Beer Distribution Game mở rộng và kịch bản mô phỏng phân phối nhu cầu thay đổi bất thường (Sudden Demand Surges):

Phương Pháp Điều Phối	Chi Phí ($)	Đứt Hàng (%)	Roi Da (σ_q^2/σ_d^2)	Độ Ổn Định
Standard Zero-Shot LLM (GPT-4)	14,820	28.4%	3.82 (Rất cao)	Kém (Dao động)
Standard CoT LLM (Chain-of-Thought)	9,640	16.2%	2.45 (Cao)	Trung bình
Heuristic Tĩnh (s, S) Policy	7,350	11.5%	1.38	Tốt ở thế tĩnh
Deep RL (PPO trained on base)	8,120	14.8%	1.62	Kém khi có sốc
LLM + Structured Safe-Stock	**5,820**	**6.1%**	**1.14**	Rất tốt
AIM-RM (Đề xuất của tác giả)	**4,910**	**3.4%**	**1.05**	Xuất sắc

Bảng 1: Bảng so sánh hiệu năng vận hành tồn kho giữa AIM-RM và các phương pháp cơ sở.

- **Khử bỏ hoàn toàn Bullwhip Effect:** Nhờ mã hóa vị thế tồn kho IP_t và truy xuất kinh nghiệm trong quá khứ, hệ số biến động đơn hàng của AIM-RM tiệm cận mức lý tưởng ($1.05 \approx 1.0$), loại bỏ triệt để hiện tượng đặt đơn hàng theo quán tính hoảng loạn.
- **Giảm 66.8% tổng chi phí** so với mô hình Zero-Shot LLM truyền thống và giảm 33.2% chi phí so với Heuristic (s, S) cố định khi đối mặt với các đột biến nhu cầu bất thường.

—

5. Ánh Xạ Chuyên Sâu Vào Kiến Trúc Dự Án Office Asset Agent

ỨNG DỤNG VÀ PHẢN BIỆN: Hiện Thực Hóa Giải Pháp Hạn Chế 2 Trong Báo Cáo Đề Tài

Trong tài liệu `docs/bao_cao_khao_sat_de_tai.tex`, đề tài đã xác định:

> *"Hạn chế 2: Thiếu tính ổn định khi ra quyết định trong chuỗi cung ứng/quản lý vật tư. Agent dễ đưa ra quyết định thất thường nếu không nhớ lịch sử cấp phát trước đó. Cách khắc phục: Cung cấp bộ nhớ tra cứu lịch sử và thiết lập quy trình duyệt cấp phát theo đúng định mức quy định của công ty."*

Paper **arXiv:2602.05524** chính là nền tảng toán học và kỹ thuật trực tiếp của module này trong hệ thống `office-asset-agent`.

Dưới đây là thiết kế module tích hợp nguyên lý AIM-RM và Structured Safe-Stock Prompts vào luồng thẩm định cấp phát và cảnh báo tồn kho của dự án:

```python
import numpy as np
from dataclasses import dataclass
from typing import List, Dict, Any, Optional

@dataclass
class AllocationEpisode:
    department: str
    item_code: str
    requested_qty: int
    historical_avg_qty: float
    current_onhand: int
    pipeline_orders: int
    is_anomaly: bool
    final_approved_qty: int
    outcome_feedback: str

class OfficeAssetMemoryRetrieval:
    """Module AIM-RM: Tra cuu lich su cap phat va tinh toan dinh muc an toan."""
    def __init__(self, db_connection):
        self.db = db_connection

    def compute_safe_stock(self, mean_demand: float, std_demand: float, lead_time_weeks: int = 1, z_score: float = 1.65) -> float:
        """Cong thuc Safe-Stock Strategy: Bao ve kho truoc bien dong chu ky."""
        return z_score * std_demand * np.sqrt(lead_time_weeks + 1)

    def retrieve_similar_episodes(self, dept: str, item_code: str, requested_qty: int, top_k: int = 3) -> List[Dict[str, Any]]:
        """Truy xuat cac ky cap phat tuong dong nhat trong lich su CSDL phong ban."""
        query = """
            SELECT department, item_code, quarter, requested_qty, approved_qty,
                   stock_status_after, notes
            FROM allocation_history
            WHERE item_code = ? AND department = ?
            ORDER BY ABS(requested_qty - ?) ASC
            LIMIT ?
        """
        cursor = self.db.cursor()
        cursor.execute(query, (item_code, dept, requested_qty, top_k))
        return cursor.fetchall()

    def build_structured_audit_prompt(self, dept: str, item_code: str, req_qty: int, onhand: int, pipeline: int, past_stats: Dict[str, float]) -> str:
        """Tao Prompt co cau truc ep LLM ra quyet dinh tham dinh an toan."""
        safe_stock = self.compute_safe_stock(past_stats['mean'], past_stats['std'])
```

```
        inventory_position = onhand + pipeline
        history = self.retrieve_similar_episodes(dept, item_code, req_qty)

        prompt = f"""[Structured Allocation Decision Framework]
1. Target Asset: {item_code} | Requesting Dept: {dept}
2. Current On-hand Inventory: {onhand} units | Pipeline Orders: {pipeline} units
3. Inventory Position (IP = Onhand + Pipeline): {inventory_position} units
4. Dynamic Safety Stock Benchmark: {safe_stock:.1f} units
5. Similar Historical Allocations for this Department:
{history}
TASK: Analyze whether requested quantity {req_qty} breaches normal consumption
    variance.
If requested quantity exceeds IP - safe_stock or historical mean by > 2 sigma,
    recommend:
FLAG_ALERT: Require supervisor manual override (Human-in-the-Loop approval)."""
        return prompt
```

Listing 1: Thiết kế module AIM-RM thẩm định cấp phát vật tư văn phòng

—

6. Bộ Câu Hỏi Phản Biện Của Hội Đồng Đồ Án & Đáp Án Chuẩn Mực

6.1 Câu Hỏi 1: Tại sao không dùng trực tiếp giải thuật quy hoạch tuyến tính (Linear Programming) hoặc EOQ cổ điển mà phải dùng Agent AI kết hợp Retrieval Memory?

Trả lời chuẩn học thuật:

- Các mô hình cổ điển như EOQ hoặc Linear Programming (LP) chỉ giải quyết bài toán tối ưu tĩnh dưới các giả định rất khắt khe: hàm mục tiêu khả vi, phân phối nhu cầu cố định, không có các yếu tố định tính và thông tin đầu vào phải hoàn toàn ở dạng số nguyên vẹn.
- Trong thực tế doanh nghiệp và quản lý văn phòng, các yêu cầu phát sinh thường chứa ngữ cảnh phi cấu trúc: lý do cấp phát đặc biệt (ví dụ: chuẩn bị đoàn kiểm toán, khai trương chi nhánh mới, hội nghị đối tác). Một giải thuật toán học thuần túy sẽ từ chối đơn hàng vì vượt định mức số học.
- Ngược lại, một LLM đơn thuần lại dễ bị ảo giác số học và bỏ qua hạn mức tồn kho an toàn.
- Mô hình AIM-RM của Yoshizato et al. kết hợp hài hòa cả hai thế mạnh: **Ràng buộc số học chặt chẽ** qua Structured Safe-Stock Prompts và **Khả năng hiểu ngữ cảnh định tính** của LLM thông qua bài học tương đồng từ Memory Retrieval.

6.2 Câu Hỏi 2: Nếu trong CSDL lịch sử chứa các quyết định cấp phát sai lầm hoặc lãng phí trong quá khứ, liệu cơ chế Retrieval Memory có vô tình khiến Agent lặp lại sai lầm đó?

Trả lời chuẩn học thuật:

- Đây là rủi ro *Suboptimal Policy Propagation*. Yoshizato et al. đã giải quyết triệt để vấn đề này bằng cơ chế **Outcome-Conditioned Filtering**:
- Mỗi episode được lưu trữ không chỉ gồm thông tin quyết định (q_k) mà còn gắn liền với nhãn phản hồi hệ quả (c_k - chi phí phát sinh và tỷ lệ thiếu hụt sau đó).

- Khi truy xuất, hệ thống chỉ lọc các kịch bản tương đồng thỏa mãn ràng buộc chất lượng: $c_k < \tau$ (hoặc có đánh giá thành công từ cán bộ quản trị). Các kịch bản dẫn đến đứt hàng hoặc dư thừa quá mức sẽ được gán nhãn *Negative Counterfactuals* và đóng vai trò như các cảnh báo răn đe trong Prompt để Agent tránh đi vào vết xe đổ.

6.3 Câu Hỏi 3: Khi áp dụng vào hệ thống Office Asset Agent, làm thế nào để đảm bảo tính an toàn dữ liệu và tuân thủ phân quyền khi Agent truy xuất lịch sử cấp phát?

Trả lời chuẩn học thuật:

- Truy vấn hồi cứu bộ nhớ không được thực hiện một cách mở tự do. Nó được tích hợp chặt chẽ với mô hình phân quyền vai trò RBAC (Sandhu et al., 1996):
- Khi nhân viên phòng Kế toán gửi yêu cầu, Agent chỉ được phép truy xuất dữ liệu lịch sử tiêu thụ của chính phòng ban đó và các định mức dùng chung do Công ty công bố.
- Chỉ tài khoản có vai trò `Admin` hoặc `Manager` mới có quyền truy xuất báo cáo chi phí tổng thể và lịch sử điều chuyển đa phòng ban. Ranh giới này được cô lập ở tầng cơ sở dữ liệu thông qua tham số hóa truy vấn SQL, đảm bảo tính bảo mật và tính toàn vẹn tuyệt đối.

—

7. Kết Luận & Định Hướng Ứng Dụng

Bài báo của Yoshizato et al. (02/2026) là một công trình mẫu mực chứng minh cách giải quyết bài toán điều phối số học phức tạp bằng LLM Agent thông qua việc **tái cấu trúc quy trình tư duy** và **cung cấp ký ức thực chứng**. Đối với dự án `office-asset-agent`, các phát hiện này cung cấp cơ sở lý thuyết vững chắc giúp hệ thống chuyển hóa từ một chatbot hội thoại thụ động thành một trợ lý điều hành kho tài sản thông minh, chuẩn xác và có khả năng tích lũy kinh nghiệm qua thời gian.